Name: Janahvi Singh
Roll No: 124A8107
Class/Div: AIDS-H2

Experiment No 12

Title :Security Analysis of AES Block Cipher Modes: ECB vs CBC

Aim: To implement the Advanced Encryption Standard (AES) using ECB and CBC modes of operation, perform encryption and decryption, and analyze the security implications of the selected block cipher mode by examining plaintext pattern visibility and ciphertext block behavior.

Course Outcome Mapping

CO2: Apply symmetric-key cryptographic techniques and mathematical concepts to design and analyze secure encryption mechanisms.

SDG Mapping

SDG 9 – Industry, Innovation and Infrastructure

The experiment develops computational and cryptographic skills required for designing secure information-processing infrastructure.

SDG 16 – Peace, Justice and Strong Institutions

The experiment demonstrates the importance of confidentiality and secure information exchange.

3. **Problem Statement :**

A plaintext containing repeated information is to be encrypted using AES under two different block cipher modes: ECB (Electronic Codebook) and CBC (Cipher Block Chaining). The experiment focuses on comparing the ciphertext blocks, verifying the correctness of decryption, and understanding the use of an Initialization Vector (IV) in CBC mode.

4. **Beyond-Syllabus Component :**

The experiment extends the basic understanding of AES modes by experimentally analyzing the security implications of ECB and CBC, including plaintext pattern leakage, comparison of ciphertext block behavior, the effect of chaining, and the role of the IV in preventing repeated ciphertext patterns.

Theory:

Advanced Encryption Standard

The **Advanced Encryption Standard (AES)** is a symmetric-key block cipher used for protecting digital information.

AES operates on blocks of:

128 bits = 16 bytes

AES supports three standard key sizes:

Key Size Number of Rounds

128 bits 10

192 bits 12

256 bits 14

In this experiment, **AES-128** is used.

The same secret key is used for encryption and decryption.

Block Cipher

A block cipher divides plaintext into fixed-size blocks and transforms each block into ciphertext using a secret key.

For AES:

Plaintext
↓
Divide into 128-bit blocks
↓
AES Encryption + Key
↓
Cipherte
xt

During decryption:

Ciphertext
↓
AES Decryption + Key
↓
Plaintex

t

ECB Mode

ECB stands for **Electronic Codebook**.

Each plaintext block is encrypted independently using the same key.

Mathematically:

$C_1 = E(K, P_1)$ $C_2 = E(K, P_2)$
$C_3 = E(K, P_3)$

Therefore, if:

$P_1 = P_3$
then:

$C_1 = C_3$
This can reveal patterns in the plaintext.

ECB Structure

P1

──AES(K)──>

C1 P2

──AES(K)──>

C2 P3

──AES(K)──>

C3

Each block operates independently.

CBC Mode

CBC stands for **Cipher Block Chaining**.

In CBC, each plaintext block is XORed with the previous ciphertext block before encryption. For the first block, an Initialization Vector (IV) is used.

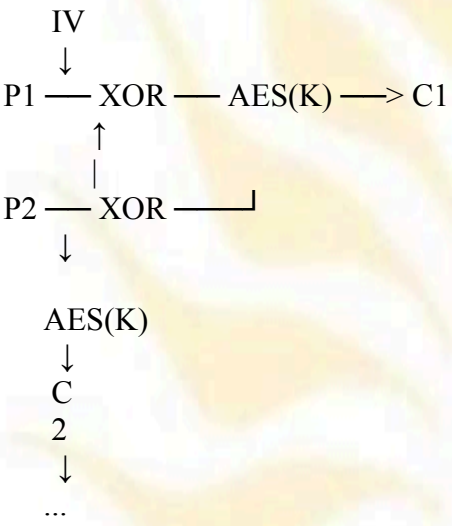
$C_1 = E(K, P_1 \text{ XOR } IV)$

$C_2 = E(K, P_2 \text{ XOR } C_1)$

$C_3 = E(K, P_3 \text{ XOR } C_2)$

Therefore, even when plaintext blocks are identical, their ciphertext blocks will generally differ because of the chaining process.

CBC Structure



Initialization Vector

An **Initialization Vector (IV)** is an additional value used with CBC mode.

The IV ensures that encryption does not start with the same internal state for every message.

In this experiment, a random 16-byte IV is generated.

The IV does **not** need to be secret, but it must be handled correctly.

Padding

AES operates on blocks of exactly 16 bytes.

If the plaintext is not a multiple of 16 bytes, padding is required.

This experiment uses **PKCS#7 padding** through the PyCryptodome library.

Requirements

Hardware

- Computer/Laptop
- Minimum 4 GB RAM recommended

Software

- Python 3.x
- Visual Studio Code
- Python extension for VS Code
- PyCryptodome library

Python Library

python -m pip install pycryptodome

Algorithm:

Algorithm for AES-ECB

1. Start.
2. Select a 128-bit AES key.
3. Enter the plaintext.

4. Apply padding to make the plaintext a multiple of 16 bytes.
5. Create an AES cipher using ECB mode.
6. Encrypt the padded plaintext.
7. Display the ciphertext.
8. Divide the ciphertext into 16-byte blocks.
9. Compare the ciphertext blocks.
10. Decrypt the ciphertext using the same key.
11. Remove padding.
12. Verify the recovered plaintext.
13. Stop.

Algorithm for AES-CBC

1. Start.
2. Select a 128-bit AES key.
3. Enter the plaintext.
4. Generate a random 16-byte IV.
5. Apply padding to the plaintext.
6. Create an AES cipher using CBC mode and the IV.
7. Encrypt the plaintext.
8. Display the IV and ciphertext.
9. Divide the ciphertext into 16-byte blocks.
10. Compare the ciphertext blocks.

11. Decrypt using the same key and IV.
12. Remove padding.
13. Verify the recovered plaintext.
14. Stop.

Code:

```
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad


# ============================================================
# AES-ECB ENCRYPTION
# ============================================================

def aes_ecb_encrypt(plaintext, key):
    cipher = AES.new(key, AES.MODE_ECB)

    padded_text = pad(
        plaintext.encode(),
        AES.block_size
    )

    ciphertext = cipher.encrypt(padded_text)

    return ciphertext


# ============================================================
# AES-ECB DECRYPTION
# ============================================================

def aes_ecb_decrypt(ciphertext, key):
    cipher = AES.new(key, AES.MODE_ECB)

    decrypted = cipher.decrypt(ciphertext)

    plaintext = unpad(
        decrypted,
        AES.block_size
    )

    return plaintext.decode()


# ============================================================
```

```python
# AES-CBC ENCRYPTION
# ============================================================

def aes_cbc_encrypt(plaintext, key):
    # Generate a random 16-byte Initialization Vector
    iv = get_random_bytes(AES.block_size)

    cipher = AES.new(
        key,
        AES.MODE_CBC,
        iv
    )

    padded_text = pad(
        plaintext.encode(),
        AES.block_size
    )

    ciphertext = cipher.encrypt(padded_text)

    return iv, ciphertext


# ============================================================
# AES-CBC DECRYPTION
# ============================================================

def aes_cbc_decrypt(iv, ciphertext, key):
    cipher = AES.new(
        key,
        AES.MODE_CBC,
        iv
    )

    decrypted = cipher.decrypt(ciphertext)

    plaintext = unpad(
        decrypted,
        AES.block_size
    )

    return plaintext.decode()
```

```
# ============================================================
# DISPLAY CIPHERTEXT BLOCKS
# ============================================================

def display_blocks(ciphertext):
    blocks = []

    for i in range(0, len(ciphertext), AES.block_size):
        block = ciphertext[i:i + AES.block_size]

        blocks.append(block)

        print(
            f"Block {i // AES.block_size + 1}: "
            f"{block.hex()}"
        )

    return blocks


# ============================================================
# MAIN PROGRAM
# ============================================================

print("=" * 70)
print("AES SECURITY ANALYSIS: ECB vs CBC")
print("=" * 70)


# AES-128 key (16 bytes)
key = b"0123456789ABCDEF"


# Repeated plaintext is deliberately used to demonstrate
# the pattern leakage problem of ECB.
plaintext = (
    "TEMP=30C;TEMP=30C:TEMP=30C:"
    "TEMP=30C;TEMP=30C;TEMP=30C;"
)

print("\nOriginal Plaintext:")
print(plaintext)
```

```
# ============================================================
# AES-ECB
# ============================================================

print("\n" + "-" * 70)
print("AES-ECB ENCRYPTION")
print("-" * 70)

ecb_ciphertext = aes_ecb_encrypt(
    plaintext,
    key
)

print("\nECB Ciphertext:")
print(ecb_ciphertext.hex())


ecb_decrypted = aes_ecb_decrypt(
    ecb_ciphertext,
    key
)

print("\nECB Decrypted Plaintext:")
print(ecb_decrypted)


print("\nECB Ciphertext Blocks:")

ecb_blocks = display_blocks(
    ecb_ciphertext
)

print("\nTotal ECB Blocks:", len(ecb_blocks))

print(
    "Unique ECB Blocks:",
    len(set(ecb_blocks))
)


# ============================================================
# AES-CBC
# ============================================================
```

```
print("\n" + "-" * 70)
print("AES-CBC ENCRYPTION")
print("-" * 70)

cbc_iv, cbc_ciphertext = aes_cbc_encrypt(
    plaintext,
    key
)

print("\nCBC Initialization Vector (IV):")
print(cbc_iv.hex())

print("\nCBC Ciphertext:")
print(cbc_ciphertext.hex())


cbc_decrypted = aes_cbc_decrypt(
    cbc_iv,
    cbc_ciphertext,
    key
)

print("\nCBC Decrypted Plaintext:")
print(cbc_decrypted)


print("\nCBC Ciphertext Blocks:")

cbc_blocks = display_blocks(
    cbc_ciphertext
)

print("\nTotal CBC Blocks:", len(cbc_blocks))

print(
    "Unique CBC Blocks:",
    len(set(cbc_blocks))
)


# ============================================================
# SECURITY ANALYSIS
# ============================================================
```

```
print("\n" + "=" * 70)
print("SECURITY ANALYSIS")
print("=" * 70)


# ECB analysis
if len(set(ecb_blocks)) < len(ecb_blocks):
    print("\nECB Observation:")
    print("Repeated ciphertext blocks are present.")
    print("This can reveal patterns in the plaintext.")
else:
    print("\nECB Observation:")
    print("No repeated ciphertext blocks were observed.")


# CBC analysis
if len(set(cbc_blocks)) == len(cbc_blocks):
    print("\nCBC Observation:")
    print("Ciphertext blocks are different.")
    print("The repeated plaintext pattern is hidden.")
else:
    print("\nCBC Observation:")
    print("Repeated ciphertext blocks were observed.")


# ============================================================
# DECRYPTION VERIFICATION
# ============================================================

print("\n" + "=" * 70)
print("DECRYPTION VERIFICATION")
print("=" * 70)


if ecb_decrypted == plaintext:
    print("ECB: Decryption successful.")
else:
    print("ECB: Decryption failed.")


if cbc_decrypted == plaintext:
    print("CBC: Decryption successful.")
else:
    print("CBC: Decryption failed.")
```

```
# ============================================================
# CONCLUSION
# ============================================================

print("\n" + "=" * 70)
print("CONCLUSION")
print("=" * 70)

print(
    "\nAES successfully encrypted and decrypted the given plaintext.\n"
    "\nECB mode can reveal patterns because identical plaintext blocks "
    "are encrypted independently.\n"
    "\nCBC mode uses an Initialization Vector (IV) and chaining, "
    "which prevents the same plaintext pattern from directly "
    "producing the same ciphertext pattern.\n"
    "\nTherefore, the experiment demonstrates that the choice of "
    "block cipher mode is an important part of cryptographic "
    "system design."
)
```

Output:

```
PS C:\Users\Dell\Desktop\CNS> python exp12A.py
======================================================================
AES SECURITY ANALYSIS: ECB vs CBC
======================================================================

Original Plaintext:
TEMP=30C;TEMP=30C:TEMP=30C:TEMP=30C;TEMP=30C;TEMP=30C;

----------------------------------------------------------------------
AES-ECB ENCRYPTION
----------------------------------------------------------------------

ECB Ciphertext:
c33ddfbac0bbe30bcb1975923c0b89544288c9838c1aa4606fba1ba82364b0b3f103aca876ca826ce5258e01e9bcd1a742efd404d4ce65447225b737b4e2d7e2

ECB Decrypted Plaintext:
TEMP=30C;TEMP=30C:TEMP=30C:TEMP=30C;TEMP=30C;TEMP=30C;

ECB Ciphertext Blocks:
Block 1: c33ddfbac0bbe30bcb1975923c0b8954
Block 2: 4288c9838c1aa4606fba1ba82364b0b3
Block 3: f103aca876ca826ce5258e01e9bcd1a7
Block 4: 42efd404d4ce65447225b737b4e2d7e2

Total ECB Blocks: 4
Unique ECB Blocks: 4

----------------------------------------------------------------------
AES-CBC ENCRYPTION
----------------------------------------------------------------------

CBC Initialization Vector (IV):
699c6c0302d9d3a8ed30b943b9e65dc7

CBC Ciphertext:
3f801d5c4e070648466adf62944b3367e041edf058ea413b4f56f84afd32022c2811fdfdcdcb48dad0a0f2dd40a0f4218d89c9048e14996a668142b1ac407009

CBC Decrypted Plaintext:
TEMP=30C;TEMP=30C:TEMP=30C:TEMP=30C;TEMP=30C;TEMP=30C;

CBC Ciphertext Blocks:
Block 1: 3f801d5c4e070648466adf62944b3367
Block 2: e041edf058ea413b4f56f84afd32022c
Block 3: 2811fdfdcdcb48dad0a0f2dd40a0f421
Block 4: 8d89c9048e14996a668142b1ac407009

Total CBC Blocks: 4
Unique CBC Blocks: 4

======================================================================
SECURITY ANALYSIS
======================================================================

ECB Observation:
No repeated ciphertext blocks were observed.

CBC Observation:
Ciphertext blocks are different.
The repeated plaintext pattern is hidden.

======================================================================
```

Activate Windows
Go to Settings to activate Windo

```
======================================================================
DECRYPTION VERIFICATION
======================================================================
ECB: Decryption successful.
CBC: Decryption successful.

======================================================================
CONCLUSION
======================================================================

AES successfully encrypted and decrypted the given plaintext.

ECB mode can reveal patterns because identical plaintext blocks are encrypted independently.

CBC mode uses an Initialization Vector (IV) and chaining, which prevents the same plaintext pattern from directly producing the sa
me ciphertext pattern.

Therefore, the experiment demonstrates that the choice of block cipher mode is an important part of cryptographic system design.
PS C:\Users\Dell\Desktop\CNS> python exp12B.py
======================================================================
```

Conclusion:

The experiment demonstrates that the security of a block-cipher-based system depends not only on the encryption algorithm but also on the mode of operation.

Although AES provides strong encryption, ECB can reveal structural patterns in repeated data. CBC introduces chaining and an IV, making ciphertext patterns less directly correlated with repeated plaintext blocks.

Thus, the experiment reinforces the importance of selecting an appropriate **block cipher mode of operation** while designing a cryptographic system.

SIES Graduate School of Technology
Sri Chandrasekarendra Saraswati Vidyapuram
Sector V, Nerul, Navi Mumbai - 400706

EXPERIMENT NO.1 2B

Design and Analysis of a Secure Encryption Scheme Using Classical Cryptography, Modular Arithmetic and AES

1. Aim

To implement and analyze classical and modern cryptographic techniques including **Caesar Cipher, Affine Cipher, Vigenère Cipher, Euclidean Algorithm, Modular Inverse, Congruence Relation, Fermat's Little Theorem, Chinese Remainder Theorem (CRT), and AES-CBC**, and verify their encryption, decryption and mathematical properties using Python.

2. Course Outcome Mapping

CO2: Apply mathematical concepts and symmetric-key cryptographic techniques to implement and analyze secure encryption mechanisms.

3. SDG Mapping

SDG 9 – Industry, Innovation and Infrastructure

The experiment develops computational and cryptographic skills required for building secure information-processing and communication infrastructure.

SDG 16 – Peace, Justice and Strong Institutions

Cryptography provides confidentiality and supports trustworthy and secure digital communication.

4. Problem Statement

A cryptographic system requires both **mathematical foundations** and **encryption techniques** to securely transform information.

Develop a Python-based cryptographic laboratory program that performs the following tasks:

1. Encrypt and decrypt a message using the **Caesar Cipher**.
2. Calculate the GCD using the **Euclidean Algorithm**.
3. Calculate a modular multiplicative inverse.
4. Encrypt and decrypt the message using the **Affine Cipher**.

5. Encrypt and decrypt the message using the **Vigenère Cipher**.
6. Verify a given **congruence relation**.
7. Verify **Fermat's Little Theorem** for suitable values.
8. Solve a system of modular equations using the **Chinese Remainder Theorem**.
9. Encrypt and decrypt the message using **AES-CBC**.
10. Compare the mathematical and cryptographic results and verify the correctness of the implemented operations.

The student must determine which mathematical conditions are necessary for the cryptographic operations to work correctly.

5. Engineering Problem / CEP Aspect

This experiment is designed as a **syllabus-based Complex Engineering Problem (CEP)**.

The problem is not simply to write individual encryption programs. Students must determine the relationship between:

Mathematical Foundations
↓
Modular Arithmetic
↓
GCD / Modular Inverse
↓
Classical Cryptographic Algorithms
↓
Symmetric-Key Encryption
↓
AES-C
BC
↓
Verification and Security Analysis

The student must identify conditions such as:

- Why must gcd(a,26) = 1 for an Affine Cipher?
- Why is a modular inverse required for decryption?
- Why must appropriate moduli be selected for CRT?
- Why is AES different from the classical substitution ciphers?
- Why does AES-CBC require an IV?

Thus, the experiment involves **analysis, implementation, verification and decision-making**, making it suitable as a CEP.

6. Theory

6.1 Caesar Cipher

The Caesar Cipher is a substitution cipher in which each alphabetic character is shifted by a fixed number.

Encryption:

$C=(P+K) \bmod 26$

Decryption:

$P=(C-K) \bmod 26$

For example, with key 3:

A → D
B → E
C → F

Therefore:

HELLO

becomes:

KHOOR

6.2 Euclidean Algorithm

The Euclidean Algorithm is used to calculate the greatest common divisor (GCD) of two integers.

For integers a and b:

$\gcd(a,b)=\gcd(b, a \bmod b)$

Example:

$\gcd(35,26)=1$

The GCD is important in cryptography because certain operations require two numbers to be relatively prime.

6.3 Modular Arithmetic

Modular arithmetic deals with remainders after integer

division. For example:

29mod26=3

Therefore:

29≡3(mod26)

Modular arithmetic is fundamental to classical cryptographic algorithms.

6.4 Modular Multiplicative Inverse

The modular inverse of a modulo m is a value a^{-1} satisfying:

a×a−1≡1(modm)

The inverse exists only when:

gcd(a,m)=1

For example:

5−1(mod26)=21

because:

5×21=105

and:

105mod26=1

6.5 Affine Cipher

The Affine Cipher combines multiplication and addition.

Encryption:

C=(aP+b)mod26

Decryption:

P=a−1(C−b)mod

26

For the cipher to be reversible:

gcd(a,26)=1

In this experiment:

a = 5
b = 8

Since:

gcd(5,26)=1

SIES GRADUATE SCHOOL OF TECHNOLOGY

the modular inverse exists.

6.6 Vigenère Cipher

The Vigenère Cipher is a poly-alphabetic substitution cipher.

Instead of using a single shift, it uses a sequence of shifts determined by a key.

For encryption:

$C_i = (P_i + K_i) \bmod 26$

For decryption:

$P_i = (C_i - K_i) \bmod 26$

For example:

Plaintext : HELLO Key : KEYKE

Each character uses a different shift.

6.7 Congruence Relation

Two integers a and b are congruent modulo m if:

$a \equiv b \pmod{m}$

when:

$m \mid (a-b)$

For

example:

29≡3(mod26

) because:

29−3=26

and 26 is divisible by 26.

6.8 Fermat's Little Theorem

If p is a prime number and a is not divisible by p, then:

SIES GRADUATE SCHOOL OF TECHNOLOGY

ap−1≡1(modp)

For example:

36mod7=1

Therefore, Fermat's Little Theorem is verified.

6.9 Chinese Remainder Theorem

The Chinese Remainder Theorem provides a method for solving simultaneous

congruences. Consider:

x≡2(mod3) x≡3(mod5) x≡2(mod7)

The solution is:

x=23

Verification:

23mod3=2 23mod5=3 23mod7=2

Therefore, the solution satisfies all three equations.

6.10 AES-CBC

AES is a modern symmetric block cipher.

AES operates on:

128-bit blocks

In this experiment AES-128 is used.

CBC stands for **Cipher Block**

Chaining. For the first block:

$C1 = EK (P1 \oplus IV)$

For subsequent blocks:

$Ci = EK (Pi \oplus Ci-1)$

CBC requires an Initialization Vector (IV).

The same secret key is used for encryption and decryption.

7. Requirements

Hardware

- Computer/Laptop
- Minimum 4 GB RAM recommended

Software

- Python 3.x
- Visual Studio Code
- Python extension for VS Code
- PyCryptodome

Python Library

Install using:

python -m pip install pycryptodome

8. Algorithm

Part A – Caesar Cipher

1. Start.
2. Read plaintext.
3. Select a shift value.
4. Convert each alphabetic character to its numerical representation.
5. Apply the Caesar encryption formula.
6. Display ciphertext.
7. Apply reverse shift for decryption.
8. Display recovered plaintext.
9. Stop.

Part B – Euclidean Algorithm

1. Read two integers a and b.
2. Check whether b is zero.
3. If not, replace a with b and b with a mod b.
4. Repeat until b = 0.
5. The final value of a is the GCD.
6. Display the result.

Part C – Modular Inverse

1. Read a and modulus m.
2. Calculate gcd(a,m).
3. If GCD is not 1, inverse does not exist.
4. Apply the Extended Euclidean Algorithm.

5. Obtain the modular inverse.

6.

Verify:

$(a \times a^{-1}) \bmod m$

$= 1$

Part D – Affine Cipher

1. Select values a and b.
2. Verify that gcd(a,26)=1.
3. Encrypt

using:

$C = (aP + b) \bmod 26$

4. Calculate the inverse of a.
5. Decrypt

using:

$P = a^{-1}(C - b) \bmod 26$

6. Compare decrypted text with original plaintext.

Part E – Vigenère Cipher

1. Read plaintext.
2. Select a keyword.
3. Repeat the keyword to match the plaintext length.
4. Convert characters to numerical values.
5. Apply the Vigenère encryption formula.
6. Display ciphertext.
7. Apply reverse shifts.
8. Verify the original plaintext.

Part F – Congruence

1. Select a, b and modulus m.
2. Calculate (a-b) mod m.
3. If the result is zero, the numbers are congruent.
4. Display the result.

Part G – Fermat's Little Theorem

1. Select a prime number p.
2. Select an integer a such that p does not divide a.

3.

Calculate:

$a^{p-1} \bmod p$

4. If the result is 1, the theorem is verified.

Part H – Chinese Remainder Theorem

1. Read the remainder values.
2. Read the corresponding moduli.
3. Calculate the product of the moduli.
4. Calculate partial products.
5. Calculate modular inverses.
6. Obtain the CRT solution.
7. Verify the solution against each congruence.

Part I – AES-CBC

1. Select a 128-bit AES key.
2. Read plaintext.
3. Generate a random IV.
4. Apply PKCS#7 padding.
5. Encrypt using AES-CBC.
6. Display IV and ciphertext.
7. Decrypt using the same key and IV.
8. Remove padding.
9. Compare decrypted text with original plaintext.

9. Procedure / Steps to Execute in VS Code

Step 1 – Check Python

Open VS Code terminal and type:

python --version

Step 2 – Install PyCryptodome

python -m pip install pycryptodome

Step 3 – Create the Python File

Create:

experiment2_cryptography.py

Step 4 – Copy the Program

Paste the Experiment 2 Python program into the file.

Step 5 – Save

Press:

Ctrl + S

Step 6 – Run

Open the VS Code terminal and execute:

python experiment2_cryptography.py

Step 7 – Observe

Observe the results of:

- Caesar Cipher

- Euclidean Algorithm
- Modular Inverse
- Affine Cipher
- Vigenère Cipher
- Congruence
- Fermat's Little Theorem
- CRT
- AES-CBC

Step 8 – Verify

Check that the decrypted messages are identical to the original plaintext.

Code:

```
# ============================================================
# EXPERIMENT 2
# DESIGN AND ANALYSIS OF A SECURE ENCRYPTION SCHEME
# ============================================================

from math import gcd

from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad


# ============================================================
# 1. CAESAR CIPHER
# ============================================================

def caesar_encrypt(text, shift):
    result = ""

    for char in text:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')

            result += chr(
                (ord(char) - base + shift) % 26 + base
            )
        else:
            result += char

    return result


def caesar_decrypt(text, shift):
    return caesar_encrypt(text, -shift)


# ============================================================
# 2. EUCLIDEAN ALGORITHM
# ============================================================

def euclidean_gcd(a, b):
    while b != 0:
        a, b = b, a % b
```

```
    return a


# ============================================================
# 3. EXTENDED EUCLIDEAN ALGORITHM
# ============================================================

def extended_gcd(a, b):
    if b == 0:
        return a, 1, 0

    gcd_value, x1, y1 = extended_gcd(b, a % b)

    x = y1
    y = x1 - (a // b) * y1

    return gcd_value, x, y


# ============================================================
# 4. MODULAR INVERSE
# ============================================================

def modular_inverse(a, m):
    gcd_value, x, y = extended_gcd(a, m)

    if gcd_value != 1:
        return None

    return x % m


# ============================================================
# 5. AFFINE CIPHER
# ============================================================

def affine_encrypt(text, a, b):
    if gcd(a, 26) != 1:
        raise ValueError(
            "Invalid key: gcd(a, 26) must be 1."
        )

    result = ""

    for char in text:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')

            x = ord(char) - base

            encrypted = (a * x + b) % 26

            result += chr(encrypted + base)
        else:
            result += char

    return result
```

```
def affine_decrypt(text, a, b):
    inverse_a = modular_inverse(a, 26)

    if inverse_a is None:
        raise ValueError(
            "No multiplicative inverse exists."
        )

    result = ""

    for char in text:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')

            y = ord(char) - base

            decrypted = (
                inverse_a * (y - b)
            ) % 26

            result += chr(decrypted + base)
        else:
            result += char

    return result


# ============================================================
# 6. VIGENERE CIPHER
# ============================================================

def vigenere_encrypt(text, key):
    key = key.upper()

    if not key.isalpha():
        raise ValueError("Vigenere key must contain only letters.")

    result = ""
    key_index = 0

    for char in text:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')

            shift = (
                ord(key[key_index % len(key)]) - ord('A')
            )

            encrypted = (
                ord(char) - base + shift
            ) % 26

            result += chr(encrypted + base)

            key_index += 1
        else:
            result += char

    return result
```

```python
def vigenere_decrypt(text, key):
    key = key.upper()

    if not key.isalpha():
        raise ValueError("Vigenere key must contain only letters.")

    result = ""
    key_index = 0

    for char in text:
        if char.isalpha():
            base = ord('A') if char.isupper() else ord('a')

            shift = (
                ord(key[key_index % len(key)]) - ord('A')
            )

            decrypted = (
                ord(char) - base - shift
            ) % 26

            result += chr(decrypted + base)

            key_index += 1
        else:
            result += char

    return result


# ============================================================
# 7. CHINESE REMAINDER THEOREM
# ============================================================

def crt(remainders, moduli):
    if len(remainders) != len(moduli):
        raise ValueError(
            "Remainders and moduli must have the same length."
        )

    product = 1

    for modulus in moduli:
        product *= modulus

    result = 0

    for remainder, modulus in zip(remainders, moduli):
        partial_product = product // modulus

        inverse = modular_inverse(
            partial_product % modulus,
            modulus
        )

        if inverse is None:
            raise ValueError(
                "CRT requires suitable coprime moduli."
            )
```

```
        result += (
            remainder *
            partial_product *
            inverse
        )

    return result % product


# ============================================================
# 8. AES-CBC
# ============================================================

def aes_encrypt(plaintext, key):
    # Generate a random 16-byte IV
    iv = get_random_bytes(AES.block_size)

    cipher = AES.new(
        key,
        AES.MODE_CBC,
        iv
    )

    padded_text = pad(
        plaintext.encode(),
        AES.block_size
    )

    ciphertext = cipher.encrypt(padded_text)

    return iv, ciphertext


def aes_decrypt(iv, ciphertext, key):
    cipher = AES.new(
        key,
        AES.MODE_CBC,
        iv
    )

    decrypted = cipher.decrypt(ciphertext)

    plaintext = unpad(
        decrypted,
        AES.block_size
    )

    return plaintext.decode()


# ============================================================
# MAIN PROGRAM
# ============================================================

print("=" * 70)
print("DESIGN AND ANALYSIS OF A SECURE ENCRYPTION SCHEME")
print("=" * 70)
```

```
# ============================================================
# CAESAR CIPHER
# ============================================================

plaintext = "HELLO WORLD"
shift = 3

caesar_ciphertext = caesar_encrypt(
    plaintext,
    shift
)

caesar_decrypted = caesar_decrypt(
    caesar_ciphertext,
    shift
)

print("\n" + "-" * 70)
print("1. CAESAR CIPHER")
print("-" * 70)

print("Plaintext :", plaintext)
print("Key       :", shift)
print("Ciphertext:", caesar_ciphertext)
print("Decrypted :", caesar_decrypted)


# ============================================================
# EUCLIDEAN ALGORITHM
# ============================================================

print("\n" + "-" * 70)
print("2. EUCLIDEAN ALGORITHM")
print("-" * 70)

a = 35
b = 26

gcd_result = euclidean_gcd(a, b)

print(f"GCD({a}, {b}) = {gcd_result}")


# ============================================================
# EXTENDED EUCLIDEAN ALGORITHM
# ============================================================

print("\n" + "-" * 70)
print("3. EXTENDED EUCLIDEAN ALGORITHM")
print("-" * 70)

a = 35
b = 26

gcd_result, x, y = extended_gcd(a, b)

print(f"GCD({a}, {b}) = {gcd_result}")
print(f"Bezout coefficients: x = {x}, y = {y}")

print(
```

```
    f"Verification: ({a} * {x}) + ({b} * {y}) "
    f"= {a * x + b * y}"
)


# ============================================================
# AFFINE CIPHER
# ============================================================

print("\n" + "-" * 70)
print("4. AFFINE CIPHER")
print("-" * 70)

a = 5
b = 8

print(f"gcd({a}, 26) =", gcd(a, 26))

inverse = modular_inverse(a, 26)

print(
    f"Multiplicative inverse of {a} modulo 26 =",
    inverse
)

affine_ciphertext = affine_encrypt(
    plaintext,
    a,
    b
)

affine_decrypted = affine_decrypt(
    affine_ciphertext,
    a,
    b
)

print("Plaintext :", plaintext)
print("Keys      :", a, b)
print("Ciphertext:", affine_ciphertext)
print("Decrypted :", affine_decrypted)


# ============================================================
# VIGENERE CIPHER
# ============================================================

print("\n" + "-" * 70)
print("5. VIGENERE CIPHER")
print("-" * 70)

vigenere_key = "KEY"

vigenere_ciphertext = vigenere_encrypt(
    plaintext,
    vigenere_key
)

vigenere_decrypted = vigenere_decrypt(
    vigenere_ciphertext,
```

```
    vigenere_key
)

print("Plaintext :", plaintext)
print("Key       :", vigenere_key)
print("Ciphertext:", vigenere_ciphertext)
print("Decrypted :", vigenere_decrypted)


# ============================================================
# CONGRUENCE RELATION
# ============================================================

print("\n" + "-" * 70)
print("6. CONGRUENCE RELATION")
print("-" * 70)

x = 29
y = 3
m = 26

print(f"{x} mod {m} =", x % m)

if (x - y) % m == 0:
    print(f"{x} = {y} (mod {m})")
else:
    print(f"{x} is not congruent to {y} modulo {m}")


# ============================================================
# FERMAT'S LITTLE THEOREM
# ============================================================

print("\n" + "-" * 70)
print("7. FERMAT'S LITTLE THEOREM")
print("-" * 70)

a = 3
p = 7

result = pow(
    a,
    p - 1,
    p
)

print(
    f"{a}^({p}-1) mod {p} =",
    result
)

if result == 1:
    print("Fermat's Little Theorem verified.")
else:
    print("Fermat's Little Theorem not verified.")


# ============================================================
# CHINESE REMAINDER THEOREM
# ============================================================
```

```
print("\n" + "-" * 70)
print("8. CHINESE REMAINDER THEOREM")
print("-" * 70)

remainders = [2, 3, 2]
moduli = [3, 5, 7]

crt_result = crt(
    remainders,
    moduli
)

print("Given equations:")
print("x = 2 (mod 3)")
print("x = 3 (mod 5)")
print("x = 2 (mod 7)")

print("\nCRT Solution: x =", crt_result)

print("\nVerification:")

for r, m in zip(remainders, moduli):
    print(
        f"x mod {m} = {crt_result % m} "
        f"(expected {r})"
    )


# ============================================================
# AES-CBC
# ============================================================

print("\n" + "-" * 70)
print("9. AES-CBC")
print("-" * 70)

# AES-128 key: exactly 16 bytes
aes_key = b"0123456789ABCDEF"

aes_iv, aes_ciphertext = aes_encrypt(
    plaintext,
    aes_key
)

aes_decrypted = aes_decrypt(
    aes_iv,
    aes_ciphertext,
    aes_key
)

print("Plaintext :", plaintext)
print("AES Key   :", aes_key.decode())
print("IV        :", aes_iv.hex())
print("Ciphertext:", aes_ciphertext.hex())
print("Decrypted :", aes_decrypted)


# ============================================================
# FINAL VERIFICATION
```

```
# ============================================================

print("\n" + "=" * 70)
print("FINAL VERIFICATION")
print("=" * 70)

tests = [
    ("Caesar Cipher", caesar_decrypted == plaintext),
    ("Affine Cipher", affine_decrypted == plaintext),
    ("Vigenere Cipher", vigenere_decrypted == plaintext),
    ("AES-CBC", aes_decrypted == plaintext)
]

all_passed = True

for test_name, test_result in tests:
    if test_result:
        print(f"{test_name}: PASS")
    else:
        print(f"{test_name}: FAIL")
        all_passed = False

if all_passed:
    print(
        "\nAll encryption/decryption tests "
        "completed successfully."
    )
else:
    print("\nSome tests failed.")


# ============================================================
# CONCLUSION
# ============================================================

print("\n" + "=" * 70)
print("CONCLUSION")
print("=" * 70)

print(
    """
The experiment demonstrates the use of classical and modern
cryptographic techniques.

Caesar, Affine and Vigenere ciphers demonstrate substitution
and poly-alphabetic encryption.

The Euclidean algorithm is used to determine GCD and supports
the calculation of modular inverses.

Congruence, Fermat's Little Theorem and the Chinese Remainder
Theorem demonstrate important mathematical foundations used
in cryptography.

AES-CBC demonstrates a modern symmetric-key encryption method.

The successful decryption confirms the correctness of the
implemented encryption and decryption procedures.
"""
)
```

Output:

```
PS C:\Users\Dell\Desktop\CNS> python exp12B.py
======================================================================
DESIGN AND ANALYSIS OF A SECURE ENCRYPTION SCHEME
======================================================================

----------------------------------------------------------------------
1. CAESAR CIPHER
----------------------------------------------------------------------
Plaintext : HELLO WORLD
Key       : 3
Ciphertext: KHOOR ZRUOG
Decrypted : HELLO WORLD

----------------------------------------------------------------------
2. EUCLIDEAN ALGORITHM
----------------------------------------------------------------------
GCD(35, 26) = 1

----------------------------------------------------------------------
3. EXTENDED EUCLIDEAN ALGORITHM
----------------------------------------------------------------------
GCD(35, 26) = 1
Bezout coefficients: x = 3, y = -4
Verification: (35 * 3) + (26 * -4) = 1

----------------------------------------------------------------------
4. AFFINE CIPHER
----------------------------------------------------------------------
gcd(5, 26) = 1
Multiplicative inverse of 5 modulo 26 = 21
Plaintext : HELLO WORLD
Keys      : 5 8
Ciphertext: RCLLA OAPLX
Decrypted : HELLO WORLD

----------------------------------------------------------------------
5. VIGENERE CIPHER
----------------------------------------------------------------------
Plaintext : HELLO WORLD
Key       : KEY
Ciphertext: RIJVS UYVJN
Decrypted : HELLO WORLD

----------------------------------------------------------------------
6. CONGRUENCE RELATION
----------------------------------------------------------------------
29 mod 26 = 3
29 = 3 (mod 26)

----------------------------------------------------------------------
7. FERMAT'S LITTLE THEOREM
----------------------------------------------------------------------
3^(7-1) mod 7 = 1
Fermat's Little Theorem verified.

----------------------------------------------------------------------
8. CHINESE REMAINDER THEOREM
----------------------------------------------------------------------
Given equations:
x = 2 (mod 3)
x = 3 (mod 5)
x = 2 (mod 7)
```

```
CRT Solution: x = 23

Verification:
x mod 3 = 2 (expected 2)
x mod 5 = 3 (expected 3)
x mod 7 = 2 (expected 2)

----------------------------------------------------------------------
9. AES-CBC
----------------------------------------------------------------------
Plaintext : HELLO WORLD
AES Key   : 0123456789ABCDEF
IV        : 58611c3fc24b58396b6404d3320b28bd
Ciphertext: 6c3ea8f868ee503a94d8921c37d341cb
Decrypted : HELLO WORLD

======================================================================
FINAL VERIFICATION
======================================================================
Caesar Cipher: PASS
Affine Cipher: PASS
Vigenere Cipher: PASS
AES-CBC: PASS

All encryption/decryption tests completed successfully.

======================================================================
CONCLUSION
======================================================================

The experiment demonstrates the use of classical and modern
cryptographic techniques.

Caesar, Affine and Vigenere ciphers demonstrate substitution
and poly-alphabetic encryption.

The Euclidean algorithm is used to determine GCD and supports
the calculation of modular inverses.

Congruence, Fermat's Little Theorem and the Chinese Remainder
Theorem demonstrate important mathematical foundations used
in cryptography.

AES-CBC demonstrates a modern symmetric-key encryption method.

The successful decryption confirms the correctness of the
implemented encryption and decryption procedures.
```

Conclusion

The experiment demonstrates that cryptography is based on both **mathematical foundations and algorithmic techniques**.

The Caesar, Affine and Vigenère ciphers demonstrate substitution-based encryption. The Euclidean algorithm and modular inverse provide the mathematical foundation required for reversible modular operations. Congruence, Fermat's Little Theorem and CRT demonstrate important concepts used in cryptographic mathematics.

AES-CBC demonstrates the transition from classical cryptography to a modern

symmetric-key block cipher.

Hence, the experiment provides an integrated understanding of the relationship between **modular mathematics, classical cryptography and modern symmetric-key encryption**.